```
 """
================================================================================
JAGAPRIYAN B
CH.SC.U4CSE24216
LAB EXERCISE 8: SUPPORT VECTOR MACHINES (SVM)

Topics Covered:
  Exercise 1: SVM on Random Quadratic Boundary Dataset (Polynomial/RBF Kernel)
  Exercise 2: Email Spam Classification based on Word Frequency & Message Length
================================================================================
"""
```

Run cell (Ctrl+Enter)
cell executed since last change

executed by Jagapriyan Boopathi
3:26 PM (0 minutes ago)
executed in 6.653s

```
s np
as pd
lib.pyplot as plt
import seaborn as sn
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

print("="*70)
print("EXERCISE 8.1: SVM ON QUADRATIC DISTRIBUTION DATASET")
print("="*70)

# Generate a 2D synthetic dataset with a quadratic boundary (y = a*x^2 + b)
np.random.seed(42)
n_samples = 300

x1 = np.random.uniform(-3, 3, n_samples)
x2 = np.random.uniform(-1, 8, n_samples)

# Quadratic decision boundary: points above y = x1^2 belong to Class 1, below to Class 0
# Add a small amount of noise
noise = np.random.normal(0, 0.4, n_samples)
y_quadratic = ((x2 > (x1**2 + noise))).astype(int)

df_quad = pd.DataFrame({'X1': x1, 'X2': x2, 'Target': y_quadratic})
print("Quadratic Dataset Sample:")
print(df_quad.head())

X_q = df_quad[['X1', 'X2']].values
y_q = df_quad['Target'].values

X_train_q, X_test_q, y_train_q, y_test_q = train_test_split(
    X_q, y_q, test_size=0.25, random_state=42
)

# Scale features
scaler_q = StandardScaler()
X_train_q_scaled = scaler_q.fit_transform(X_train_q)
X_test_q_scaled = scaler_q.transform(X_test_q)

# Train SVM with RBF and Polynomial Kernels
svm_rbf = SVC(kernel='rbf', C=1.0, random_state=0)
svm_rbf.fit(X_train_q_scaled, y_train_q)

y_pred_q = svm_rbf.predict(X_test_q_scaled)
print(f"SVM (RBF Kernel) Accuracy: {accuracy_score(y_test_q, y_pred_q):.4f}")
print("Confusion Matrix:\n", confusion_matrix(y_test_q, y_pred_q))

# Plotting the Decision Boundary for the Quadratic Dataset
def plot_svm_decision_boundary(model, X, y, title):
    h = 0.05
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', edgecolors='k')
    plt.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1],
                s=80, facecolors='none', edgecolors='k', linewidths=1.5, label='Support Vectors')
    plt.title(title)
    plt.xlabel("X1 (Standardized)")
    plt.ylabel("X2 (Standardized)")
    plt.legend()
    plt.show()
```

```
plot_svm_decision_boundary(svm_rbf, X_train_q_scaled, y_train_q, "SVM with RBF Kernel on Quadratic Distribution")


print("\n" + "="*70)
print("EXERCISE 8.2: EMAIL SPAM CLASSIFICATION USING SVM")
print("="*70)
# Features:
# 1. word_freq_free (frequency of word 'free' or offer words)
# 2. word_freq_win (frequency of word 'win' or claim words)
# 3. message_length (character / word count)
# Target: 1 (Spam), 0 (Not Spam / Ham)
```

Run cell (Ctrl+Enter)
cell executed since last change

executed by Jagapriyan Boopathi
3:26 PM (0 minutes ago)
executed in 6.653s

```
(42)
lower promo words, normal length
= np.random.exponential(scale=0.2, size=n_emails//2)
ham_freq_win = np.random.exponential(scale=0.1, size=n_emails//2)
ham_length = np.random.normal(loc=120, scale=40, size=n_emails//2).clip(20, 500)
ham_labels = np.zeros(n_emails//2, dtype=int)

# Spam emails: high promo words, varied/longer lengths
spam_freq_free = np.random.exponential(scale=3.5, size=n_emails//2) + 0.5
spam_freq_win = np.random.exponential(scale=2.8, size=n_emails//2) + 0.3
spam_length = np.random.normal(loc=280, scale=80, size=n_emails//2).clip(40, 800)
spam_labels = np.ones(n_emails//2, dtype=int)

df_emails = pd.DataFrame({
    'word_freq_free': np.concatenate([ham_freq_free, spam_freq_free]),
    'word_freq_win': np.concatenate([ham_freq_win, spam_freq_win]),
    'message_length': np.concatenate([ham_length, spam_length]),
    'is_spam': np.concatenate([ham_labels, spam_labels])
}).sample(frac=1, random_state=42).reset_index(drop=True)

print("Spam Dataset Sample:")
print(df_emails.head())
print(f"Spam vs Ham counts:\n{df_emails['is_spam'].value_counts()}")

X_email = df_emails[['word_freq_free', 'word_freq_win', 'message_length']]
y_email = df_emails['is_spam']

X_train_e, X_test_e, y_train_e, y_test_e = train_test_split(
    X_email, y_email, test_size=0.3, random_state=42
)

scaler_e = StandardScaler()
X_train_e_scaled = scaler_e.fit_transform(X_train_e)
X_test_e_scaled = scaler_e.transform(X_test_e)

# Train SVM Classifier (linear and rbf kernel comparisons)
svm_spam = SVC(kernel='linear', C=1.0, random_state=0)
svm_spam.fit(X_train_e_scaled, y_train_e)
y_pred_e = svm_spam.predict(X_test_e_scaled)

acc_spam = accuracy_score(y_test_e, y_pred_e)
conf_spam = confusion_matrix(y_test_e, y_pred_e)

print(f"\nLinear SVM Spam Detection Accuracy: {acc_spam:.4f} ({int(acc_spam*100)}%)")
print("Confusion Matrix:\n", conf_spam)
print("\nClassification Report:")
print(classification_report(y_test_e, y_pred_e, target_names=['Ham (0)', 'Spam (1)']))

# Confusion Matrix Visual
plt.figure(figsize=(5, 4))
sn.heatmap(conf_spam, annot=True, fmt='d', cmap='Oranges',
           xticklabels=['Ham', 'Spam'], yticklabels=['Ham', 'Spam'])
plt.title("SVM Email Spam Classification Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Run cell (Ctrl+Enter)
cell executed since last change

executed by Jagapriyan Boopathi
3:26 PM (0 minutes ago)
executed in 6.653s